AI-Powered Earnings Call Analysis Using an End-to-End AI Pipeline

Author: Abiodun Joy

Hackathon: SeqHub AI Research Hackathon

Date: July 2026

1. Introduction

Earnings calls provide valuable insight into a company's financial performance, business strategy, and future outlook, but manually reviewing lengthy audio recordings is time-consuming and inefficient. This project presents an end-to-end AI pipeline that automatically analyzes earnings call recordings by converting speech into structured, machine-readable information. The pipeline integrates multiple AI models to perform transcription, speaker diarization, topic segmentation, boundary detection, sentiment analysis, and JSON export. The modular design allows each processing stage to operate and be verified independently while contributing to a unified analysis workflow.

2. Project Objective

The primary objective of this project is to develop an automated AI pipeline that can transform raw earnings call audio into structured insights. Specifically, the system aims to:

- Convert speech into accurate text through automatic transcription.
- Identify and separate individual speakers throughout the conversation.
- Detect the main discussion topics within the transcript.
- Identify points where the discussion shifts from one topic to another.
- Analyze the sentiment expressed during each speaker's turn.
- Export the processed results into a structured JSON format for downstream applications and analysis.

3. System Architecture

Earnings Call Audio

↓

Faster-Whisper
(Transcription)

↓

Pyannote Audio
(Speaker Diarization)

↓

BERTopic Model
(Topic Segmentation)

↓

Boundary Detection

↓

Transformer Sentiment Model

↓

Structured JSON Output

The system follows a sequential processing pipeline where the output of one module becomes the input to the next. This modular architecture simplified development and testing, as each component could be verified against the ground truth in isolation before being trusted to feed the next stage. Additionally, individual components could be updated or replaced without requiring redesign of the entire system.

4. Technologies Used

Technology	Purpose
Python	Core programming language
Faster-Whisper	Speech transcription
Pyannote Audio	Speaker diarization
BERTopic + Sentence-Transformers	Topic segmentation

scikit-learn	Stop-word filtered keyword extraction
Hugging Face Transformers	Sentiment analysis
jiwer	Word Error Rate (WER) evaluation
matplotlib	Data visualization
JSON	Structured output format
Google Colab	Development environment

5. Methodology

5.1 Transcription

The audio recording is processed using the faster-whisper "base" model, which converts spoken language into timestamped text. The test earnings call (approximately 160 seconds) was transcribed into 60 timestamped segments. The resulting transcript forms the foundation for all subsequent analysis stages.

5.2 Speaker Diarization

Pyannote Audio is used to identify different speakers within the recording. The initial diarization run under-detected speakers, identifying only 3 voices instead of the 4 known speakers (CEO, CFO, and two analysts), likely due to acoustic similarity between two speakers. Since the true speaker count was known in advance, it was passed explicitly to the pipeline (num_speakers=4), which corrected the clustering and produced all 4 distinct speakers across 44 detected speaker turns.

5.3 Merging Transcript and Speaker Labels

Since Whisper and Pyannote produce independent segment boundaries, each transcript segment was aligned to the diarization turn with which it shares the greatest time overlap. Consecutive segments from the same speaker were then merged into full speaker turns (17 turns from the original 60 fragments), giving later stages richer, more complete text to work with.

5.4 Topic Segmentation

The speaker-aligned transcript is processed using BERTopic to group related conversations into discussion topics. An initial attempt on the raw 60 fragments performed poorly; 40% of segments could not be confidently assigned a topic, and the resulting keywords were dominated by filler words. Working on the 17 merged speaker turns, and filtering common English stop words during keyword extraction, produced substantially more meaningful topics (e.g., "SMB churn," "cost/acquisition/renewals"), discovering 3 broad topic clusters.

5.5 Boundary Detection

Topic boundaries are identified by comparing each turn's assigned topic to the previous turn's. Whenever the topic changes, a boundary is flagged, dividing the transcript into logical sections. This approach detected 11 boundaries across the 17 speaker turns, correctly keeping single-topic discussions (such as the SMB churn segment) grouped across multiple turns.

5.6 Sentiment Analysis

Each speaker turn is analyzed using a transformer-based sentiment classification model (cardiffnlp/twitter-roberta-base-sentiment-latest), predicting whether the expressed sentiment is positive, neutral, or negative. The model performed with high confidence on clearly emotional statements (e.g. 97% confidence on the CEO's positive opening) but tended to classify cautious, measured business language as neutral rather than mildly negative.

5.7 Evaluation (Word Error Rate)

To quantitatively validate the transcription stage, Word Error Rate (WER) was computed using the jiwer library, comparing the pipeline's transcript against the ground truth reference text. The result, 0.0958 (9.6%), confirms the transcription's strong qualitative performance and is consistent with two specific errors identified through manual review: "CAC" misheard as "CSE," and "TechVenture" split into two words.

5.8 JSON Export

The outputs generated from each stage of the pipeline are combined into a structured JSON file, including transcription, speaker labels, detected topics, topic boundaries, sentiment predictions, and the computed WER score, in a machine-readable format suitable for downstream use.

6. Challenges Encountered

Several real challenges arose during development. Managing library dependencies in Google Colab required careful, iterative version resolution; particularly recurring binary incompatibilities between NumPy, PyTorch, Torchaudio, and

Pyannote Audio, including a CUDA-version mismatch after a forced library upgrade. GPU usage limits were also exhausted during testing, prompting consideration of Kaggle as an alternative environment. Beyond environment issues, aligning speaker labels and improving topic quality required genuine iterative refinement: diarization under-counted speakers until given an explicit speaker-count hint, and topic segmentation on raw fragments performed poorly until segments were merged into fuller turns with stop words filtered from keyword extraction.

7. Results

The completed pipeline successfully performs transcription, speaker diarization, topic segmentation, boundary detection, sentiment analysis, quantitative WER evaluation, and structured JSON export. On the test earnings call, the system produced 60 transcribed segments merged into 17 full speaker turns across 4 correctly identified speakers, discovered 3 topic clusters with 11 detected boundaries, and achieved a Word Error Rate of 9.6%. The generated visualizations (speaker distribution, topic timeline, and sentiment distribution) provide an intuitive summary of speaker participation, discussion topics, and emotional tone, making the results easier to interpret at a glance.

Stage	Status	What Worked	What Didn't	Evidence
Transcription	Strong	Accurate on financial terms (ARR, EBITDA); 9.6% WER	Missed "CAC" (heard as "CSE"); "TechVenture" split into two words	60/60 segments produced, cross-checked against ground truth text
Diarization	Fixed	num_speakers=4 hint corrected under-clustering	Default run only found 3 of 4 speakers	3→4 speakers after fix; 44 speaker turns detected
Topic Segmentation	Moderate	Merging turns + stop-word filtering produced meaningful clusters (churn, acquisition cost, etc.)	Found 3 broad topics vs. 8 in ground truth	Outlier rate dropped from 40% (fragment-level) to ~18% (turn-level)
Boundary Detection	Moderate	Correctly held single topics together across multiple turns (e.g. SMB churn discussion)	Under-segments relative to ground truth's finer topics	11 boundaries detected across 17 turns
Sentiment Analysis	Strong	High confidence on clearly emotional statements	Reads cautious language as neutral, not mildly negative	97% confidence on CEO's positive opening; correctly neutral on clarifying questions

8. Key Tradeoffs

- Whisper "base" model: faster and lighter, at the cost of some accuracy (e.g. the CAC/CSE mishearing) ; a larger model would improve accuracy but run slower.
- Merging fragments into full speaker turns before topic modeling: gained far better topic quality, at the cost of losing sentence-level granularity within a turn.
- Deriving topic boundaries directly from topic IDs rather than a separate model: kept the pipeline simple, at the cost of boundary accuracy being fully dependent on topic clustering quality.

9. Assumptions

- The number of speakers (4) was known in advance from the problem statement and used to configure diarization (num_speakers=4).
- Whisper's "base" model size was chosen as a speed/accuracy balance appropriate for the hackathon's timeframe; a larger model would likely reduce transcription errors further.
- Topic labels are unsupervised and auto-generated, since the system has no prior knowledge of what topics a given call will cover.
- A transcript segment's speaker is determined by maximum time overlap with a diarization turn; no segments in this test run were left unassigned.

10. Future Improvements

Future work could focus on improving transcription accuracy by comparing larger Whisper model sizes, completing the remaining quantitative metrics (DER, NMI, C_v, WindowDiff/Pk) to fully cover all five pipeline stages, refining topic segmentation to better match fine-grained ground-truth topic categories, incorporating multilingual support, and developing an interactive dashboard for visual exploration of results beyond static charts.

11. Conclusion

This project demonstrates the successful development of an end-to-end AI pipeline for automated earnings call analysis. By integrating multiple AI models into a modular, independently-verified workflow, the system transforms unstructured audio into structured business insights, while honestly documenting the real limitations and design tradeoffs encountered along the way. The project highlights the practical application of modern speech processing and natural language processing techniques for analyzing financial communications.